INF-2200: Datamaskinarkitektur og organisering

Assignment 1

Jens Christian Valen Leynse

Jørgen Angelo Lyngstad

September 14th, 2018

1 Introduction

The given assignment focuses on comparing the two coding languages c and assembly. A micro-benchmark is to be selected for the implementation. The next task is to identify the hotspots of the code and improve on the performance in this area by implementing the necessary function as a function in x86 assembly.

1.1 Requirements

- Select a relevant, realistic and reliable micro-benchmark
- Identify the hotspots of the code where most of the processing time is spent
- Implement “main loop” as a function in x86 assembly.
- Write C-code that:
  - initializes necessary data structures for your function.
  - calls your function.
  - tests the results for correctness

2 Technical Background

The benchmark chosen for the assignment is the mergesort algorithm. The advantage of this algorithm being that the written code is simple and easy to read. It also has a decent complexity of O(n log(n)) for both the best and the worst case scenario. The algorithm is also compatible with x86 assembly.

In coding language there are three layers of languages. The topmost layer is normal code and is easily written and read by programmers. The third layer is binary code and is constructed by ones and zeros and is therefore only read by the computer. The second layer is called assembly and can be considered as human readable binary. Assembly uses registers for storage in order to avoid unnecessarily accessing the memory. By avoiding the memory the computer will use fewer clock cycles and as a result spend less time processing the code.

3 Design & Implementation

The implementation of the mergesort algorithm only uses two functions. Function one is calling on the function mergesort recursively and splits the array into smaller arrays. This is done through two functions using the different halves after separation. The next function merge then takes these array's and combines them in a sorted order. The merge function sorts by comparing the split arrays and then combines them by putting the smaller element to the left and the larger to the right.

The alternative to merge is written in a different mergesort function where the function itself does the same, but then sends the split array into the assembly function for the sorting instead of the c-function merge.

The main function is implemented with a loop to create a random array which is sent to the different sorting functions. To make it easier to understand the process several print lines are included to keep track of the process. Even if the profiling is done with gprof, and alternative timing method is also implemented which uses the time library [1] to double check with the compiling time.

5 Discussion

The mergesort algorithm can only handle up to one million random numbers before being unable to compile. This being the case when the entire mergesort algorithm was written in x86 assembly.

The two versions of the merge function were each given an array with one million numbers. The compiler used about 0.2 seconds with the c-code, while the assembly code sorted the same amount in 0.0007 seconds.

We personally thought this improvement seemed like it was too much of an improvement and we have considered that our assembly code never truly sorted the array. We were never able to use print functions to print the array so we could check whether or not the array was actually sorted. There is a good possibility that the assembly code doesn't sort anything and the time we got (0.0007) is only the time it took to run through the code without doing anything.

When we first wrote the assembly code, the entire mergesort algorithm was implemented in assembly. This was done to get a better understanding of how the algorithm would be written in the language. Unfortunately, the code would only compile correctly when the code was like this. When the code was shortened to only include the merge function it would no longer

compile without causing a segmentation fault. When messing around with some print testing the code sometimes compiled correctly, leaving us with some data to compare, but no concrete solution was found by the time of our hand in.

When compiling and running the program we were advised to use a program to measure the time spent in hotspots, this program was called gprof. When running the program through gprof we struggled with understanding how to read the data we were given in the gprof file. However, the results show some difference between the processing time with, and without the assembly implementation. Leading us to understand that the time spent in mergesort c-code was longer than the spent in mergesort assembly code.

6 Conclusion

The two different implementations using the mergesort algorithm as a benchmark shows that the assembly code compiles faster than the code written in c. The merge algorithm didn't have a lot of functions, so the profiler could only find a hotspot in the merge function. But the performance time comparison uses the overall time of the function to compare, so the code simply improves on the entire function instead of only a section.

References

[1] Time library

https://www.tutorialspoint.com/c_standard_library/time_h.htm